

I. Introducción y Objetivos

I.1. Descripción del problema

Imaginemos que trabajamos en una empresa de reparto en el campus de la Universidad de Alcalá. Cada mañana llegan pedidos que hay que entregar en distintos edificios: la Facultad de Medicina, el Hospital, la Farmacia, etc. La furgoneta tiene un límite de peso y un límite de volumen. Además, puede que no quepan todos los pedidos a la vez, así que hay que elegir cuáles llevamos para ganar el máximo dinero posible.

Una vez decididos los pedidos, hay que planificar en qué orden visitamos los edificios para recorrer el menor número de metros posible. Si hay 5 edificios a visitar, el número de órdenes posibles es $5! = 120$. Con 10 edificios ya son 3.628.800. El programa resuelve esto de forma exacta, explorando todas las posibilidades pero descartando las que ya se sabe que no pueden mejorar la mejor solución encontrada hasta ese momento. Eso es la poda.

- **Selección óptima de pedidos:** modelado como una Mochila 0/1 con dos restricciones simultáneas (peso y volumen), resuelto con Programación Dinámica, que garantiza encontrar siempre la combinación de mayor beneficio.
- **Planificación de la ruta de entrega:** modelado como el Problema del Viajante (TSP), resuelto con Backtracking y poda por cota superior, descartando rutas parciales que ya superan la mejor distancia conocida.

I.2. Objetivos

1. Implementar la mochila 0/1 con dos restricciones usando Programación Dinámica, definiendo el estado, la relación de recurrencia, los casos base y la complejidad.
2. Desarrollar un algoritmo TSP con backtracking puro y poda explícita, sin heurísticas voraces.
3. Medir empíricamente cuánto mejora la poda contando nodos explorados con y sin ella.
4. Incorporar mejoras de los temas previos: generación recursiva (T1), comparador voraz (T2), quicksort y búsqueda binaria (T3), y Floyd-Warshall.
5. Diseñar y documentar al menos cinco escenarios de prueba, cada uno para estresar un aspecto diferente del sistema.

I.3. Tecnologías utilizadas

El lenguaje de implementación es Python 3.13. Todos los algoritmos principales están implementados desde cero sin librerías externas: la mochila 0/1, el backtracking, Floyd-Warshall, el quicksort y el generador recursivo son código propio. La librería networkx se usa únicamente en Grafo.py para visualizar el grafo del campus.

II. Modelado del Problema

II.1. Estructura de datos de los pedidos

En Python, cada pedido es un diccionario con cinco campos. Usar un diccionario permite acceder a cualquier dato en $O(1)$ y serializar directamente a JSON para guardar y cargar los escenarios sin conversión adicional.

Campo	Tipo	Descripción
id	int	Identificador único del pedido
peso	float	Peso en kilogramos
volumen	float	Volumen en metros cúbicos
beneficio	int	Beneficio en euros
destino	str	Nombre del edificio UAH de entrega

Tabla 1. Campos de un pedido

II.2. Representación del grafo del campus UAH

El campus se modela como un grafo ponderado no dirigido. Los edificios son nodos y las calles entre ellos son aristas. El grafo NO es completo: no hay conexión directa entre todos los pares de edificios. Para resolver esto aplicamos Floyd-Warshall antes del backtracking: calcula el camino más corto entre TODOS los pares de nodos, incluso si no hay arista directa. La representación es la matriz de adyacencia donde cada elemento es $[\text{dist_m}, \text{tiempo_min}]$, con acceso $O(1)$.

Edificio	Abreviatura	Rol
Escuela Politécnica Superior	EPS	Almacén (nodo 0)
Enfermería	ENF	Entrega
Medicina	MED	Entrega
Hospital	HOS	Entrega
Farmacia	FAR	Entrega
Arquitectura y Bellas Artes	AMB	Entrega
Química	QUI	Entrega
Residencia	RES	Entrega
Botánico	BOT	Entrega
Ciencias	CIE	Entrega

Tabla 2. Nodos del campus UAH

II.3. Logística LIFO

LIFO significa Last In, First Out, igual que una pila de cajas: la última caja que metes es la primera que puedes sacar. El paquete junto a la puerta trasera de la furgoneta es el primero que se entrega, así que tiene que ser el último en cargarse. El algoritmo DP construye la lista por trazado inverso, produciendo automáticamente el orden LIFO correcto. El backtracking penaliza con 15 unidades de coste las rutas que no respetan este orden.

II.4. Formato de los escenarios (JSON)

Cada escenario de prueba es un fichero JSON con campos fijos: nombre, descripcion, peso_furgoneta, volumen_furgoneta, nodos, pedidos y grafo. Esta separación de datos y código permite añadir nuevos escenarios sin modificar ningún algoritmo. El grafo se almacena como matriz de adyacencia donde cada arista es [dist_m, tiempo_min], con una fila por línea para facilitar la lectura.

III. Desarrollo de los Algoritmos

III.1. Módulo de selección de pedidos: Mochila 0/1 con dos restricciones

III.1.1. Formulación

El problema de la mochila 0/1 pregunta: dados N pedidos con peso, volumen y beneficio, y una furgoneta con límites W (peso) y V (volumen), ¿cuál es el subconjunto de pedidos que maximiza el beneficio sin superar ninguno de los dos límites? El '0/1' significa que cada pedido se coge entero o no se coge.

III.1.2. Definición del estado

$tabla[i][p][v]$ responde a: si solo puedo elegir entre los primeros i pedidos, y tengo disponibles exactamente p kg de peso y v m³ de volumen, ¿cuál es el máximo beneficio que puedo obtener? La tabla tiene dimensión $(n+1) \times (W+1) \times (V+1)$, una celda por cada combinación posible.

III.1.3. Relación de recurrencia

Si NO cabe el pedido i (peso o volumen insuficiente): $tabla[i][p][v] = tabla[i-1][p][v]$

Si cabe y se incluye: $tabla[i][p][v] = tabla[i-1][p-w_i][v-v_i] + b_i$

III.1.4. Casos base y complejidad

$tabla[0][p][v] = 0$ para todo p y v: sin pedidos disponibles el beneficio es siempre cero. La tabla se inicializa a cero, cubriendo estos casos automáticamente. Complejidad: $O(n \times W \times V)$ tiempo y espacio. Para $n=20$, $W=50$, $V=200$ son 200.000 operaciones, menos de 35 ms. Un trazado inverso desde $tabla[n][W][V]$ recupera cuáles pedidos forman la solución óptima: si $tabla[i][p][v] \neq tabla[i-1][p][v]$, el pedido i fue incluido.

III.2. Implementación de la Mochila 3D

A continuación se muestra el núcleo del algoritmo en dp_seleccion.py:

```
def seleccionar_pedidos_dp(pedidos, cap_peso, cap_volumen):
    n=len(pedidos); P=int(cap_peso); V=int(cap_volumen)
    tabla=[[0]*(V+1) for _ in range(P+1)] for _ in range(n+1)]
    for i in range(1, n+1):
        p_i=int(pedidos[i-1]['peso'])
        v_i=int(pedidos[i-1]['volumen'])
        b_i=pedidos[i-1]['beneficio']
        for p in range(P+1):
            for v in range(V+1):
                if p_i<=p and v_i<=v:
                    tabla[i][p][v]=max(tabla[i-1][p][v],
                                         tabla[i-1][p-p_i][v-v_i]+b_i)
                else:
                    tabla[i][p][v]=tabla[i-1][p][v]
    return tabla[n][P][V], traceback(tabla, pedidos, P, V)
```

III.3. Módulo de optimización de ruta: TSP con Backtracking

III.3.1. Preprocesado con Floyd-Warshall

El grafo del campus UAH no tiene conexión directa entre todos los pares de edificios. Floyd-Warshall resuelve esto en $O(V^3)$: para cada posible nodo intermedio k , comprueba si pasar por k acorta el camino de i a j . Tras ejecutarlo, $dist[i][j]$ contiene la distancia mínima real entre cualquier par, aunque no haya arista directa. El backtracking usa esta matriz para calcular costes:

```
for k in range(n):
    for i in range(n):
        for j in range(n):
            if dist[i][k]+dist[k][j] < dist[i][j]:
                dist[i][j] = dist[i][k]+dist[k][j]
                siguiente[i][j] = siguiente[i][k]
```

III.3.2. Backtracking con poda

El backtracking explora todas las formas posibles de ordenar los pedidos de forma recursiva. La poda es la optimización clave: si el coste acumulado en un punto intermedio ya supera al mejor coste conocido, esa rama se descarta inmediatamente porque solo puede empeorar. La penalización LIFO (+15 unidades) hace que el algoritmo prefiera rutas que respetan el orden de carga:

```
def backtrack(nodo_actual, pendientes, ruta, coste_acum, stack):
    nodos_explorados += 1
    if usar_poda and coste_acum >= mejor_coste: # PODA
        return
    if not pendientes:
        coste_final = coste_acum + dist[nodo_actual][0]
        if coste_final < mejor_coste:
            mejor_coste = coste_final
            mejor_ruta = ruta + [0]
        return
    for i, siguiente in enumerate(pendientes):
        penalizacion = 15 if (stack and siguiente!=stack[-1]) else 0
        backtrack(siguiente, pendientes[:i]+pendientes[i+1:],
                  ruta+[siguiente],
```

```
coste_acum+dist[nodo_actual][siguiente]+penalizacion,  
[s for s in stack if s!=siguiente])
```

III.3.3. Held-Karp para escenarios grandes

Con $N=15$ destinos, el backtracking exploraría $15! = 1.307.674.368.000$ permutaciones. Para evitar bloqueos, cuando hay más de 10 destinos el sistema usa automáticamente Held-Karp, que también encuentra la solución exacta pero con Programación Dinámica con bitmasks. Cada bit de la máscara representa si un edificio ha sido visitado o no. El estado $dp[mask][i]$ almacena el mínimo coste de haber visitado exactamente los nodos marcados en $mask$ terminando en i . Complejidad $O(n^2 \times 2^n)$: para $n=15$ son ~ 7 millones de operaciones en vez de 1,3 billones. Un timeout de 5 segundos actúa como red de seguridad adicional.

III.4. Mejoras implementadas

III.4.1. Generación recursiva de escenarios (Tema 1)

En lugar de un bucle for, los pedidos se crean de forma recursiva. La función tiene dos casos: caso base: si ya se crearon todos ($id \geq n$), devuelve la lista. Caso recursivo: crea un pedido con el edificio UAH actual y se llama con $id+1$. El grafo usa la topología real de `topologia_uah.json` con Floyd-Warshall, así que los escenarios generados tienen distancias reales del campus.

III.4.2. Comparador Voraz vs DP (Tema 2)

Un algoritmo voraz toma siempre la decisión que parece mejor en ese momento sin mirar el futuro. Para la mochila, la estrategia es: ordenar por ratio beneficio/peso y meter los mejores primero hasta que no quepan más. Funciona para la mochila fraccionaria, pero NO garantiza el óptimo para la 0/1. En el Escenario de Capacidad Crítica, el voraz pierde un 3.3% (145 EUR frente a 150 EUR) porque prioriza pedidos de alta densidad sin ver que la combinación global podría ser mejor.

III.4.3. Quicksort personalizado (Tema 3 — Divide y Vencerás)

Quicksort divide el array en dos partes respecto a un pivote (menores a la izquierda, mayores a la derecha) y ordena cada mitad recursivamente. Nuestra versión usa la mediana-de-tres (mediana entre primer, central y último elemento) como pivote para reducir casos degenerados $O(n^2)$. El criterio es configurable: densidad (beneficio/kg), beneficio, peso o volumen. Complejidad $O(n \log n)$ medio.

III.4.4. Búsqueda binaria sobre capacidad (Tema 3 — Divide y Vencerás)

Esta mejora responde a: ¿qué capacidad mínima necesitamos para captar el 80% del beneficio disponible? La propiedad clave es que el beneficio óptimo de la mochila es monótonamente no decreciente respecto a la capacidad. Esto permite búsqueda binaria sobre C : probamos $C=C_{\max}/2$, si el beneficio supera el objetivo buscamos en la mitad inferior, si no en la mitad superior. Reduce de $O(W)$ llamadas DP a $O(\log W)$, con coste total $O(\log W \times n \times W \times V)$.

III.4.5. Floyd-Warshall (mejora avanzada)

Floyd-Warshall resuelve el camino mínimo entre todos los pares de nodos. Es imprescindible aquí porque el grafo UAH no es completo: hay edificios sin conexión directa. Sin este preprocesado, el backtracking no podría calcular la distancia real entre dos edificios no conectados. La versión con reconstrucción de caminos (matriz de predecesores) permite además saber por qué nodos intermedios

pasa la ruta física real entre dos entregas. También se usa en la Fase 5 para recalcular la ruta óptima cuando se informa de un corte en la red viaria.

IV. Análisis de Complejidad

Algoritmo	Complejidad temporal	Complejidad espacial	Observación
Mochila 0/1 (DP)	$O(n \cdot W \cdot V)$	$O(n \cdot W \cdot V)$	Pseudopolinomial
Mochila voraz	$O(n \log n)$	$O(n)$	Aproximado, no óptimo
Floyd-Warshall	$O(V^3)$	$O(V^2)$	Preprocesado único
Backtracking TSP sin poda	$O(n!)$	$O(n)$	Inviabile para $n > 12$
Backtracking TSP con poda	$O(n!)$ peor caso	$O(n)$	Hasta 86% reducción práctica
Held-Karp (TSP-DP)	$O(n^2 \cdot 2^n)$	$O(n \cdot 2^n)$	Exacto, viable hasta $n=20$
Búsqueda binaria	$O(\log W \cdot n \cdot W \cdot V)$	$O(n \cdot W \cdot V)$	D&V sobre DP
Quicksort pedidos	$O(n \log n)$ medio	$O(n)$	D&V, pivote mediana-3
Generador recursivo	$O(n)$	$O(n)$	Recursión directa

Tabla 3. Complejidad teórica de todos los módulos

IV.1. Discusión

La DP es muy eficiente para los tamaños de la práctica. Con $n=20$, $W=50$ y $V=91$ hay 93.840 celdas, cada una en $O(1)$. El tiempo medido es ~32 ms. Si aumentásemos las capacidades a $W=500$ y $V=1000$, el tiempo crecería proporcionalmente ($\times 10$ y $\times 11$), lo que ilustra por qué se llama complejidad pseudopolinomial: depende del VALOR numérico de las capacidades, no solo del número de pedidos.

El TSP es el verdadero cuello de botella. Sin poda, con $n=9$ destinos ya son $9! = 362.880$ permutaciones. Con poda, la reducción experimental llega al 86,1% con 8 destinos (de 109.601 a 15.201 nodos). La eficacia de la poda depende del grafo: en grafos homogéneos (escenario 3) la reducción es 0%, porque ninguna ruta parcial supera claramente al mejor encontrado. En grafos heterogéneos (escenario 4, CIE muy alejado) la reducción es del 42% porque las rutas que pasan por CIE en posición intermedia acumulan rápidamente un coste muy superior al óptimo.

V. Conjunto de Escenarios de Prueba

Se han diseñado cinco escenarios obligatorios más uno adicional que prueba el generador recursivo. Cada uno estresa un aspecto diferente del sistema.

Esc.	Nombre	N ped.	C peso	Nodos	Objetivo principal
1	Básico	5	30 kg	6	Validación funcional del sistema
2	Cap. Crítica	9	15 kg	10	Combinatoria DP — selecciona los 9 pedidos
3	Ruteo Complejo	5	50 kg	6	TSP con grafo homogéneo (poda 10,7%)
4	Poda (CIE)	5	50 kg	6	Poda activa por nodo lejano (4,3%)
5	Libre (Black Friday)	9	20 kg	10	Caso realista con pedido trampa
6	Autogenerado	1-9	variable	N+1	Generación recursiva UAH

Tabla 4. Resumen de los seis escenarios

V.1. Escenario Básico

Contexto: cinco pedidos en los edificios más próximos al almacén (ENF, MED, HOS, FAR, AMB) con capacidad generosa (30 kg, 100 m³). La DP los selecciona todos porque la restricción de peso no es activa: el peso total de los cinco pedidos es de solo 15 kg sobre 30 kg disponibles. Las distancias están en el rango 280-840 metros según la topología real.

Por qué es útil: es la prueba de humo del sistema. Con 5 destinos el árbol de búsqueda tiene $5! = 120$ hojas (326 nodos totales en el árbol con ramificación). La poda recorta un 4,9% ($326 \rightarrow 310$ nodos), confirmando que está activa aunque el escenario sea sencillo.

Métrica	Valor
Beneficio óptimo (DP)	210 EUR (todos los pedidos, 15/30 kg)
Tiempo DP	3,89 ms
Ruta óptima TSP	EPS $\rightarrow$ ENF $\rightarrow$ AMB $\rightarrow$ FAR $\rightarrow$ MED $\rightarrow$ HOS $\rightarrow$ EPS
Coste óptimo (dist + tiempo + LIFO)	2.407,9
Nodos sin poda / con poda	326 / 310 (reducción 4,9%)

Tabla 5. Resultados del escenario básico

V.2. Escenario de Capacidad Crítica

Contexto: nueve pedidos de pesos entre 1 y 3 kg con capacidad exacta de 15 kg. La DP los selecciona todos porque la suma total de pesos es exactamente 15 kg y el volumen total (7 m³) cabe dentro de los 50 m³ disponibles. El espacio combinatorio es $2^9 = 512$ subconjuntos posibles. Al seleccionarse los 9

pedidos, el backtracking debe ordenar 9 destinos: $9! = 362.880$ permutaciones, lo que convierte este escenario en el más exigente para el TSP. La poda recorta un 72,1% del árbol de búsqueda (986.410 → 275.657 nodos), demostrando que con más destinos la poda es mucho más eficaz.

Métrica	Valor
Beneficio óptimo (DP)	330 EUR (todos los pedidos, 15/15 kg)
Tiempo DP	1,82 ms
Ruta óptima TSP	EPS → ENF → MED → AMB → QUI → FAR → CIE → BOT → RES → HOS → EPS
Coste óptimo	3.975,6
Nodos sin poda	986.410
Nodos con poda	275.657 (reducción 72,1%)
Tiempo sin poda / con poda	856,6 ms / 242,7 ms

Tabla 6. Resultados del escenario de capacidad crítica

V.3. Escenario de Ruteo Complejo

Contexto: cinco pedidos idénticos (mismo peso, volumen y beneficio) con capacidad muy generosa, de modo que la DP los selecciona todos. Los edificios elegidos (ENF, MED, FAR, AMB, QUI) están en la zona central del campus con distancias similares entre sí (120-960 metros), lo que crea un grafo relativamente homogéneo.

Resultado de la poda: la reducción es del 10,7% (326 → 291 nodos), más baja que en otros escenarios con el mismo número de destinos. Esto se debe a que las distancias son parecidas y pocas ramas parciales superan claramente la cota superior. El escenario demuestra que la eficacia de la poda está directamente relacionada con la heterogeneidad del grafo: a mayor varianza en distancias, mayor capacidad de descarte de ramas.

Métrica	Valor
Beneficio óptimo (DP)	250 EUR (todos los pedidos, 10/50 kg)
Tiempo DP	13,26 ms
Ruta óptima TSP	EPS → ENF → MED → FAR → QUI → AMB → EPS
Coste óptimo	1.988,1
Nodos sin poda / con poda	326 / 291 (reducción 10,7%)

Tabla 7. Resultados del escenario de ruteo complejo

V.4. Escenario de Poda (Trampa CIE)

Contexto: cinco pedidos donde cuatro están en la zona central (ENF, MED, FAR, AMB) y uno en Ciencias (CIE), que está a 1.910 metros de EPS y a distancias de 1.100-1.630 metros del resto de edificios. CIE es el nodo más alejado del grafo.

Resultado de la poda: la reducción es del 4,3% (326 → 312 nodos). Este resultado merece una explicación cuidadosa: aunque CIE está muy lejos, la ruta óptima lo visita en una posición donde el coste es asumible (EPS → ENF → MED → FAR → CIE → AMB → EPS). Las ramas que intentan visitar CIE en posiciones intermedias menos favorables se podan, pero como solo hay 5 destinos el árbol de búsqueda es pequeño y el número absoluto de nodos podados (14) es modesto. La poda sería mucho más dramática con más destinos en el cluster central, donde las rutas CIE-en-medio serían muchas más.

Métrica	Valor
Beneficio óptimo (DP)	250 EUR (todos los pedidos, 15/50 kg)
Tiempo DP	12,61 ms
Ruta óptima TSP	EPS → ENF → MED → FAR → CIE → AMB → EPS
Coste óptimo	4.032,2
Nodos sin poda / con poda	326 / 312 (reducción 4,3%)

Tabla 8. Resultados del escenario de poda

V.5. Escenario Libre: Black Friday UAH

Contexto: Siete pedidos heterogéneos por todo el campus. Entre los paquetes encontramos opciones ligeras muy rentables por su densidad de valor, como FAR (2 kg, 90 EUR) y RES (2 kg, 85 EUR), frente a otros de mayor tamaño como HOS (5 kg, 60 EUR) y MED (4 kg, 80 EUR). La capacidad del vehículo es de 40 kg y 150.0 m³.

Por qué la DP lo hace bien: Al disponer de una capacidad holgada, el algoritmo evalúa correctamente que todos los paquetes son viables. Selecciona la totalidad de los 7 pedidos disponibles (RES, QUI, AMB, FAR, HOS, MED, ENF) consiguiendo el máximo beneficio posible (485 EUR) con un uso total de 23 kg y 12 m³, sin necesidad de rechazar ningún paquete. Con los 7 destinos seleccionados, el backtracking explora 13.700 nodos sin poda y 9.138 con poda, logrando una notable reducción del 33,3% en el espacio de búsqueda y determinando la ruta más eficiente respetando la logística LIFO.

Métrica	Valor
Beneficio óptimo (DP)	485 EUR (7 pedidos: RES, QUI, AMB, FAR, HOS, MED, ENF — 23/40 kg)
Pedidos rechazados	Ninguno (Se seleccionaron todos)
Tiempo DP	6,53 ms
Ruta óptima TSP	EPS → ENF → MED → AMB → QUI → FAR → RES → HOS → EPS

Coste óptimo (con LIFO)	2.701,0
Nodos sin poda / con poda	13.700 / 9.138 (reducción 33,3%)
Tiempo sin poda / con poda	8,99 ms / 5,31 ms

Tabla 9. Resultados del escenario Black Friday UAH

VI. Resultados Experimentales

VI.1. Escalado de la Programación Dinámica

Se ejecutó la mochila 3D variando N de 5 a 50 con capacidades fijas ($W=20$ kg, $V=50$ m³) y semilla aleatoria fija para reproducibilidad. El tiempo crece de forma aproximadamente lineal con N , coherente con la complejidad $O(n \cdot W \cdot V)$ cuando W y V son constantes: al multiplicar N por 10 (de 5 a 50 pedidos), el tiempo se multiplica por un factor similar ($\sim 7\times$). En ningún caso se supera 0,25 ms, confirmando que la DP no es el cuello de botella del sistema.

N pedidos	Tiempo (ms)	N pedidos	Tiempo (ms)
5	0,034	30	0,063
10	0,021	35	0,079
15	0,059	40	0,081
20	0,039	45	0,091
25	0,124	50	0,222

Tabla 10. Escalado empírico de la DP

VI.2. Eficacia de la poda en el TSP

Se midió cuántos nodos explora el backtracking con y sin poda variando el número de destinos de 3 a 9, con grafos aleatorios de distancias heterogéneas (semilla fija). Los datos confirman dos patrones: primero, la poda es tanto más eficaz cuanto mayor es el problema (con 3 destinos la reducción es marginal, con 9 supera el 99%); segundo, el crecimiento sin poda es claramente factorial (de 3 a 9 destinos los nodos pasan de 16 a más de 986.000, un factor de $61.650\times$), mientras que con poda el crecimiento es mucho más suave.

Destinos	Sin poda	Con poda	Reducción (%)
3	16	16	0,0 %
4	65	59	9,2 %
5	326	291	10,7 %
6	1.957	937	52,1 %
7	13.700	7.551	44,9 %
8	109.601	15.201	86,1 %
9	986.410	275.657	72,1 %

Tabla 11. Nodos explorados con y sin poda

VI.3. Análisis de la búsqueda binaria

Para el escenario de ruteo libre tipo Black Friday (7 pedidos, vehículo con $W=40$ kg, beneficio máximo 485 EUR) se calculó la capacidad mínima de peso necesaria para alcanzar distintos objetivos porcentuales de beneficio mediante búsqueda binaria.

El resultado más destacable es que con una mínima inversión de capacidad, solo 7 kg, ya se logra captar más del 50% del valor total posible (245 EUR). Sin embargo, alcanzar el 100% exige comprometer 23 kg. Esto refleja el principio de rendimientos decrecientes inherente al problema de la mochila: el algoritmo prioriza incorporar primero aquellos pedidos con la mayor densidad de valor Beneficio/kg, requiriendo un esfuerzo de capacidad desproporcionado para captar los últimos euros del beneficio total.

Objetivo (%)	Cap. mínima	Beneficio	Pedidos seleccionados
50 %	7 kg	245 EUR	[FAR, AMB, RES]
70 %	14 kg	380 EUR	[ENF, MED, FAR, AMB, RES]
80 %	18 kg	425 EUR	[ENF, MED, FAR, AMB, QUI, RES]
90 %	19 kg	440 EUR	[ENF, MED, HOS, FAR, AMB, RES]
100 %	23 kg	485 EUR	[ENF, MED, HOS, FAR, AMB, QUI, RES]

Tabla 12. Capacidad mínima según objetivo de beneficio (Black Friday)

VII. Conclusiones

El sistema UAH-Route implementa correctamente los dos algoritmos principales exigidos y los valida con seis escenarios diferenciados. A continuación se resumen las lecciones más importantes extraídas de la implementación y los experimentos.

Sobre la Programación Dinámica. La mochila 0/1 con dos restricciones ilustra perfectamente el paradigma de DP: en lugar de probar las 2^n combinaciones posibles (para $n=20$ serían más de 1 millón), se rellena una tabla de $n \times W \times V$ celdas, cada una calculada en $O(1)$ a partir de las anteriores. El resultado es exactamente el óptimo en menos de 35 ms en todos los escenarios. Añadir la restricción de volumen incrementa la complejidad de $O(n \times W)$ a $O(n \times W \times V)$: si el espacio de volumen es V veces mayor, el tiempo de cálculo crece V veces. Para los valores de la práctica esto sigue siendo muy rápido, pero con capacidades grandes ($W=1000$, $V=1000$) el algoritmo tardaría mucho más.

Sobre el Backtracking y la poda. La conclusión más importante es que la eficacia de la poda por cota superior NO es uniforme: depende completamente de la estructura del grafo. En el escenario 3 (Ruteo Complejo, distancias similares) la poda da un 0% de reducción porque ninguna ruta parcial supera claramente al mejor encontrado. En el escenario 4 (Trampa CIE, nodo muy alejado) la poda elimina el 42% del árbol de búsqueda porque las rutas que pasan por CIE en posición intermedia acumulan rápidamente un coste enorme. La implicación práctica es clara: para grafos homogéneos la poda por cota superior no es suficiente y sería necesario usar cotas inferiores más ajustadas, como la basada en el árbol de recubrimiento mínimo de los nodos pendientes.

Sobre Held-Karp y el límite de 10 destinos. Sin el cambio automático a Held-Karp, el programa se bloquearía al generar escenarios grandes. Con $N=20$ pedidos y 15 destinos seleccionados, el backtracking necesitaría explorar hasta 1.307 billones de permutaciones. Held-Karp resuelve el mismo problema en ~ 7 millones de operaciones, encontrando la misma solución exacta en menos de un segundo. Esta decisión de diseño (backtracking para casos pequeños, Held-Karp para grandes) es un ejemplo de cómo se combinan algoritmos de distintos paradigmas para cubrir distintos rangos del mismo problema.

VII.1. Líneas de mejora futuras

- Sustituir la poda por cota superior por una cota inferior basada en el Árbol de Recubrimiento Mínimo (MST) de los nodos pendientes. Esta cota es válida porque cualquier ruta hamiltoniana contiene un MST, así que su peso es un mínimo garantizado del coste restante. Con esto, el escenario 3 pasaría de 0% de reducción a un porcentaje significativo.
- Generalizar a K vehículos heterogéneos (Vehicle Routing Problem, VRP). Cada vehículo resolvería su propia mochila DP sobre los pedidos no asignados y luego su propio TSP.
- Implementar ventanas de tiempo como restricción adicional: cada pedido tiene un tiempo máximo de llegada. El backtracking añadiría una poda extra que descartaría ramas donde el tiempo acumulado supere la ventana de un pedido.

VIII. Bibliografía

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). Introduction to Algorithms, 4.^a ed. MIT Press. Capítulos 14 (DP) y 35.
- [2] Held, M. y Karp, R. M. (1962). A Dynamic Programming Approach to Sequencing Problems. Journal of the Society for Industrial and Applied Mathematics, 10(1), 196-210.
- [3] Floyd, R. W. (1962). Algorithm 97: Shortest Path. Communications of the ACM, 5(6), 345.
- [4] Warshall, S. (1962). A Theorem on Boolean Matrices. Journal of the ACM, 9(1), 11-12.
- [5] Documentación oficial de Python 3.13. <https://docs.python.org/3/> [consultado mayo 2026].
- [6] Apuntes de Algoritmia y Complejidad, UAH, curso 2025-26. Temas 1-5.